

Home Linking on Campaign Creation

Full technical sheet for FLYR iOS/backend provisioning. Updated May 11, 2026 with parcel bridge guidance.

Short answer: yes. For automatic home-to-building linking during campaign creation/provisioning, the canonical linker is `StableLinkerService.runSpatialJoin()`. Parcels should be treated as first-class creation-time evidence whenever available, not just later repair data.

1. Executive Summary

- **One automatic creation linker:** `StableLinkerService.runSpatialJoin()`. Manual correction and parcel repair wrap the same model.
- **Creation is two-phase:** iOS inserts a campaign shell, saves the polygon, then calls backend provision.
- **Provision is Gold-first:** municipal/Gold data wins when available; Lambda/S3 backs the Silver path.
- **Parcels are the Gold bridge:** when parcel data is available, pass it into the stable linker during provision and rank it above pure nearest-building matching.
- **Current code already supports creation-time parcel input:** provision prepares parcels before linking and passes ready parcels to `runSpatialJoin()`. The product direction should make that non-optional for supported regions.
- **Old SQL gold linker is legacy:** `link_campaign_addresses_gold` exists in migrations, but current provision uses TypeScript stable linker flow.

2. Ownership Map

Concern	Owner / File	Notes
Campaign shell creation	iOS - <code>CampaignsAPI.createV2</code>	Inserts campaign metadata only. Address JSON is ignored.
Territory polygon	iOS - <code>updateTerritoryBoundary</code>	Writes GeoJSON polygon to <code>campaigns.territory_boundary</code> .
Provision trigger	iOS - <code>provisionCampaign</code>	POST <code>/api/campaigns/provision</code> with <code>campaign_id</code> and bearer token.
Source resolution	Backend provision route + <code>GoldAddressService</code>	Gold-first; otherwise Lambda/S3.
Building normalization	<code>BuildingAdapter</code>	Gold rows and Lambda GeoJSON become the same feature shape.
Creation linker	<code>StableLinkerService.runSpatialJoin</code>	Canonical address-to-building linker.
Parcel bridge	<code>ParcelEnrichmentService</code> + <code>StableLinkerService</code>	Creation-time evidence when ready; repair/relink fallback when not ready.
Manual fixes	<code>StableLinkerService.assign/unassign</code> methods	Scoped one-address operations, no full campaign relink.

3. End-to-End Creation Flow

```
iOS NewCampaignScreen
-> CampaignsAPI.createV2(payload)
    inserts campaign shell only
-> CampaignsAPI.updateTerritoryBoundary(campaign_id, polygon)
    writes campaigns.territory_boundary
-> CampaignsAPI.provisionCampaign(campaign_id)
    POST /api/campaigns/provision

Backend provision route
-> validates user/campaign access
-> loads territory_boundary and region
-> Gold-first address/building source resolution
```

```

-> inserts campaign_addresses
-> stores campaign_snapshots metadata
-> runCampaignPostProcessing inline or deferred
    -> BuildingAdapter.fetchAndNormalize(...)
    -> ParcelEnrichmentService.prepareParcelsForProvision(...)
    -> StableLinkerService.runSpatialJoin(..., parcels)
    -> TownhouseSplitterService.processCampaignTownhouses(...)
    -> quality/map-mode assessments

```

4. Backend Provision Details

- **Request:** POST /api/campaigns/provision with campaign_id. Optional body controls include geometry_tier, allow_gold, linker_mode, append_addresses, and quick-start center/radius metadata.
- **Access:** authenticated owner can provision; workspace owner/admin can also provision workspace campaigns.
- **Precondition:** campaigns.territory_boundary must exist. Missing polygon returns a provision error.
- **Source:** backend checks reusable snapshot metadata, then runs GoldAddressService.getAddressesForPolygon. If Gold is not selected, Lambda/S3 supplies snapshot data.
- **Address loading:** addresses are normalized, deduplicated, and bulk inserted into campaign_addresses. A zero-address result returns a 422 provision error.
- **Map readiness:** Lambda/Silver can mark map_ready before post-processing completes. For parcel-supported regions, the preferred A+ behavior is to run parcel-aware linking inline or report clearly that link optimization is still pending.

5. Parcel Bridge: A+ Evidence Policy

For detached homes, parcels are the difference between "nearby" and "belongs to this property." Address points often land on a driveway, lawn, road edge, or slightly offset from the footprint; the parcel can still identify the correct structure.

Rank	Evidence	Quality	Notes
1	Address point inside building footprint	Excellent	Direct containment. Keep as top evidence.
2	Address point and building centroid/footprint are inside the same parcel	Excellent	Treat as a hard bridge when parcel geometry is trusted. Current linker emits parcel_verified at 0.95.
3	Address point inside parcel and there is one main building on that parcel	Excellent	Use parcel ownership plus main-building selection. Prefer largest/non-outbuilding structure.
4	Point on building surface/boundary	Strong	Good for boundary precision issues, but parcel can be more meaningful for offset points.
5	Nearest building only	Okay/risky	Use as fallback after parcel evidence is unavailable or ambiguous.

- **Implementation direction:** keep one linker, but make parcels a first-class input to that linker during creation/provision.
- **Current code path:** runCampaignPostProcessing calls prepareParcelsForProvision() and passes ready parcels into runSpatialJoin().
- **Gap to close:** the current internal tier order checks point-on-surface before parcel bridge. For A+ detached-home accuracy, same-parcel evidence should rank above pure boundary/proximity evidence and below direct containment.
- **Operational policy:** if parcel region is supported and parcel prep succeeds, creation/provision should produce parcel-aware links before declaring final link quality. If deferred, mark the campaign as map-ready but link-optimization-pending.
- **Repair path remains valid:** if parcels are unavailable at creation time, parcel enrichment can rerun the same linker later. That should be a fallback, not the preferred path.

6. When the Linker Runs

Case	Behavior	A+ expectation
Gold source	Post-processing runs inline with rethrowOnError true.	Run parcel-aware linker inline when parcels are supported.
Lambda/Silver source, default	Post-processing may be deferred through Next after(...).	Do not treat parcel bridge as only repair; run before final optimized/link-quality state.
Lambda/Silver with force stable	Runs inline when geometry_tier=diamond, linker_mode=stable, or allow_gold=true.	Good debug/test path; could become default for parcel-supported regions.
Parcel enrichment repair	Can rerun StableLinkerService later with parcel bridge inputs.	Fallback when parcels were missing, stale, or failed during creation.

7. StableLinkerService Anatomy

- **Input:** campaign ID, normalized building GeoJSON, Overture release, optional parcels, reset flag, and persistence mode.
- **Address fetch:** reads all campaign addresses with id, gers_id, formatted, house_number, street_name, geom.
- **Building prep:** filters tiny/noise buildings and infers missing building street names from contained addresses.
- **Parcel prep:** prepared parcels keep buildings whose centroids fall inside each parcel; address points inside that parcel can bridge to those buildings.
- **Reset:** clears previous links/slices/orphans when requested; Gold mode also clears address-side link fields.
- **Output:** saves matches, saves orphans, detects multi-unit buildings, and returns SpatialJoinSummary.

Current tier	Match type	Rule	Confidence
1	containment_verified / containment_suspect	Address point inside building polygon; street score decides verified vs suspect.	1.0 / 0.85
2	point_on_surface	Address point is on polygon boundary/surface.	0.9 / 0.85
3	parcel_verified	Address point and building candidate share a parcel bridge.	0.95
4	proximity_verified / proximity_fallback	Nearest candidates within 50m, ranked by street score, distance, and area.	0.5-0.9
5	proximity_fallback	Nearest unused building within 100m when no stronger evidence exists.	>= 0.5

8. Persistence Contract

Table / Column	Written by	Purpose
campaign_addresses	Provision address load	Holds the homes/leads for the campaign.
campaign_snapshots	Provision route	Stores S3 bucket/key/url/count metadata for Lambda/Silver geometry.
building_address_links	StableLinkerService.saveMatches	Main address-to-building link table. Upsert key: campaign_id,address_id.
address_orphans	StableLinkerService.saveOrphans	Unlinked or ambiguous addresses needing review.
building_units	TownhouseSplitterService	Multi-unit/townhouse unit geometry.
campaign_addresses.building_id	Gold mode + manual paths	Direct public building assignment used heavily by Gold features.
campaign_addresses.match_source/confidence	Gold mode + manual paths	Source and confidence visible to feature APIs/UI.

- The link table has a unique index on campaign_id, address_id, so one campaign address cannot be attached to multiple buildings at the same time.

- Buildings can still hold many addresses, which is needed for townhomes, apartments, and multi-unit footprints.

9. Gold vs Lambda/Silver

Aspect	Gold	Lambda/Silver
Building source	Municipal/Gold rows from Supabase-side Gold services.	S3 snapshot GeoJSON returned by Lambda.
Normalization	BuildingAdapter.fromGoldRows; building gers_id becomes Gold row id.	BuildingAdapter.fromLambdaGeoJSON; preserves Lambda properties.
Linker used	StableLinkerService.runSpatialJoin.	StableLinkerService.runSpatialJoin.
Parcel bridge	Use if parcel region supported and prep is ready.	Use if parcel region supported and prep is ready; avoid leaving it only to later repair.
Address-side update	Yes: building_id, match_source, confidence.	No automatic address-side Gold assignment; relies mainly on building_address_links and feature joins.
Timing	Usually inline during provision.	Often deferred after map_ready unless forced.

10. Manual Correction and Repair Paths

- **POST** /api/campaigns/[campaignId]/buildings/[buildingId]/addresses links one address to one building.
- **DELETE** on the same route unlinks an address, optionally deleting manual addresses.
- Manual operations instantiate StableLinkerService and call scoped assign/unassign methods. They do not rerun the full campaign join.
- ParcelEnrichmentService.relinkCampaign can rerun the same stable linker later with parcels. This is useful repair, but creation-time parcel bridging is preferred whenever available.

11. Operational Checks

Question	Where to check
Did provision start?	campaigns.provision_status and provision_phase.
Did addresses load?	campaign_addresses count for campaign_id.
Which source won?	campaigns.provision_source: gold or lambda.
Were parcels available?	parcel_enrichment_status, has_parcels, parcel_count / related metadata.
Did parcel bridge contribute?	building_address_links.match_type = parcel_verified.
Did automatic links get created?	building_address_links where campaign_id = ...
Did Gold write direct assignments?	campaign_addresses.building_id, match_source, confidence.
Are there orphans/conflicts?	address_orphans status pending_review or ambiguous_match.
Was post-processing deferred?	provision_phase map_ready with low/zero link confidence, then later optimized.

12. Debug Checklist

- If a detached home links to a neighbor, inspect whether parcel data was available and whether the final match came from proximity instead of parcel_verified.
- If a new campaign shows homes but no linked buildings, check whether post-processing was deferred and whether it later reached optimized.
- If Gold buildings appear unlinked, inspect campaign_addresses.building_id, match_source, and confidence.

- If Silver buildings appear unlinked, inspect `building_address_links` first, then feature API joins.
- If link coverage is below 95 percent, inspect orphans and source geometry quality.
- If many addresses are orphaned in dense areas, the density guard may have forced containment-only matching.

13. Source References

iOS create shell: `FLYR/Features/Campaigns/CampaignsAPI.swift:224-318`

iOS save territory: `FLYR/Features/Campaigns/CampaignsAPI.swift:803-822`

iOS provision trigger: `FLYR/Features/Campaigns/CampaignsAPI.swift:903-980`

Create screen orchestration: `FLYR/Features/Campaigns/Views/NewCampaignScreen.swift:360-470`

Provision route: `backend-api-routes/app/api/campaigns/provision/route.ts:845-1135`

Creation-time parcel pass: `backend-api-routes/app/api/campaigns/provision/route.ts:702-750`

Stable linker entry: `backend-api-routes/lib/services/StableLinkerService.ts:161-335`

Stable linker tiers: `backend-api-routes/lib/services/StableLinkerService.ts:412-558`

Parcel bridge implementation: `backend-api-routes/lib/services/StableLinkerService.ts:814-928`

Stable linker persistence: `backend-api-routes/lib/services/StableLinkerService.ts:1253-1316`

Manual link methods: `backend-api-routes/lib/services/StableLinkerService.ts:1320-1468`

Schema - building links: `supabase/migrations/20250206000000_create_building_tables.sql:86-175`

Unique one-link constraint:

`supabase/migrations/20260504120000_enforce_one_building_link_per_campaign_address.sql:1-80`

Legacy SQL gold linker: `supabase/migrations/20260220200002_gold_linker.sql:1-90`

Backend contract: `docs/BACKEND_API_CONTRACT.md:1-130`

Provision architecture: `docs/ARCHITECTURE_PROVISION.md:1-80`